Exp No: 29

PRIORITY-BASED INTERRUPT HANDLING

AIM:

To model priority-based interrupt servicing.

SOFTWARE REQUIRED:

• Ubuntu (or WSL Ubuntu)

• g++

• CMake

• SystemC

PROGRAM:

```
#include <systemc.h>
SC_MODULE(priority_interrupt) {
sc_event low_int, high_int;
void low_ISR() {
while(true) {
wait(low_int);
cout << "Low Priority Interrupt at "
<< sc_time_stamp() << endl;
wait(3, SC_SEC);
}
}
void high_ISR() {
while(true) {
wait(high_int);
cout << "High Priority Interrupt at "

<< sc_time_stamp() << endl;
wait(1, SC_SEC);
```

```
}
}

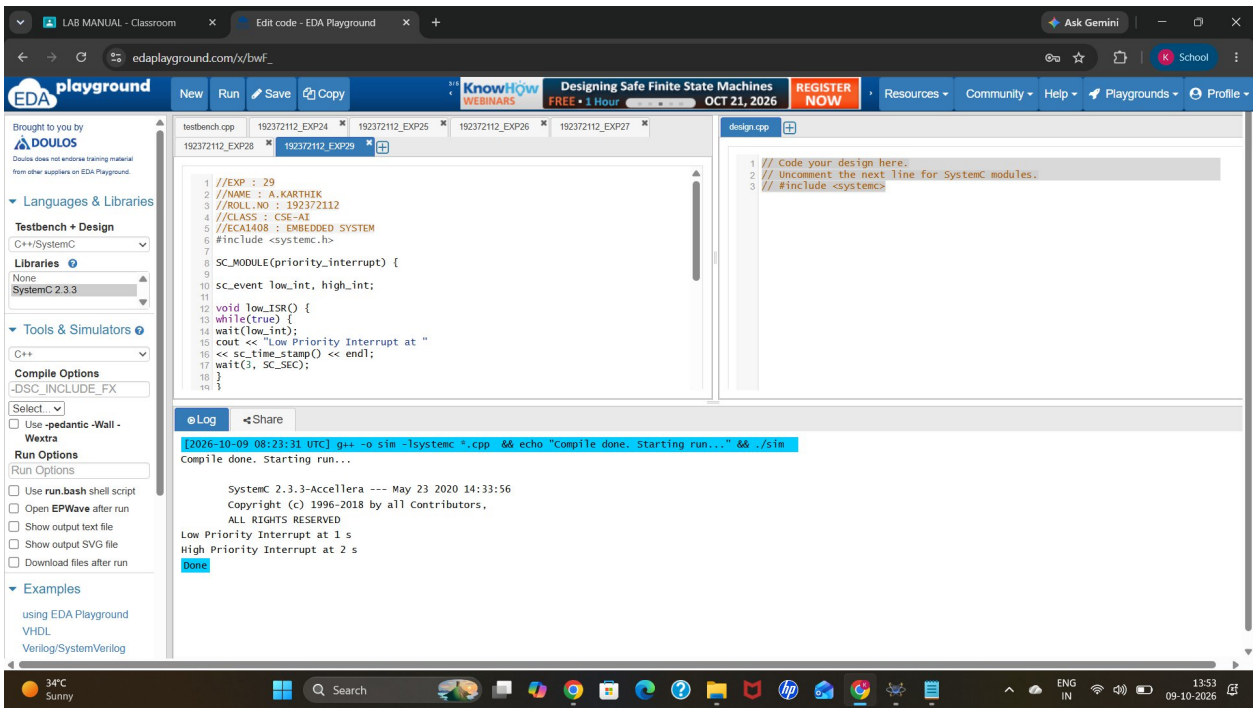
void generator() {
wait(1, SC_SEC);
low_int.notify();

wait(1, SC_SEC);
high_int.notify();
}

SC_CTOR(priority_interrupt) {
SC_THREAD(low_ISR);
SC_THREAD(high_ISR);
SC_THREAD(generator);
}
};

int sc_main(int argc, char* argv[]) {
priority_interrupt obj("Priority_INT");
sc_start(10, SC_SEC);
return 0;
}
```

OUTPUT:



RESULT:

Thus the Priority based Interrupt handling was executed using SystemC and verified.